

Task 1: Check SELinux Status

Check the current SELinux mode:

- `getenforce`

Expected Output:

- `enforcing`

Get detailed information:

- `sestatus`

Task 2: Switch SELinux to Enforcing Mode

If SELinux is in Permissive mode:

- `sudo setenforce 1`

Verify:

- `getenforce`

Task 3: Simulate an SELinux Access Denial

Create a test directory:

- `sudo mkdir -p /tmp/testapp`

Create a sample web page:

- `echo "<h1>SELinux Test</h1>" | sudo tee /tmp/testapp/index.html`

Assign an incorrect SELinux context:

- `sudo chcon -t user_home_t /tmp/testapp`
- `sudo chcon -t user_home_t /tmp/testapp/index.html`

Verify the context:

- `ls -Zd /tmp/testapp`
- `ls -Z /tmp/testapp/index.html`

Check the SELinux denial logs:

- `sudo ausearch -m AVC -ts recent`

Task 4: Assign the Correct SELinux Context

Change the context to the correct web content type:

- `sudo chcon -R -t httpd_sys_content_t /tmp/testapp`

Verify:

- `ls -Zd /tmp/testapp`
- `ls -Z /tmp/testapp/index.html`

Restart Apache:

- `sudo systemctl restart httpd`

Open the page in a browser:

- `http://localhost/`

Task 5: Documentation

Commands Used

- `getenforce`
- `sestatus`
- `sudo setenforce 1`
- `getenforce`
- `sudo mkdir -p /tmp/testapp`
- `echo "<h1>SELinux Test</h1>" | sudo tee /tmp/testapp/index.html`
- `sudo chcon -t user_home_t /tmp/testapp`
- `sudo chcon -t user_home_t /tmp/testapp/index.html`
- `ls -Zd /tmp/testapp`
- `ls -Z /tmp/testapp/index.html`
- `sudo ausearch -m AVC -ts recent`
- `sudo grep denied /var/log/audit/audit.log`
- `sudo chcon -R -t httpd_sys_content_t /tmp/testapp`
- `sudo systemctl restart httpd`

Brief Explanation

- SELinux controls access using security contexts (labels). When the web server tried to access a file labeled `user_home_t`, SELinux denied the request because that label is not permitted for web content. Changing the label to `httpd_sys_content_t` allowed Apache to read the files while maintaining SELinux protection.